

平成 2 8 年度

筑波大学大学院博士課程
システム情報工学研究科
コンピュータサイエンス専攻

博士前期課程（一般入学試験 8 月期）

試験問題 基礎科目（情報基礎）

Fundamentals of Computer Science

[注意事項][Instructions]

1. 試験開始の合図があるまで、問題の中を見てはいけません。また、筆記用具を手に持ってはいけません。
Do NOT open this booklet before the examination starts. In addition, do not have a pen in hand.
2. 試験開始の合図のあとで、全ての解答用紙の定められた欄に、研究科、専攻、受験番号を記入すること。
Fill in the designated spaces on each answer sheet with the name of the graduate school, name of your main (major) field, and the examination number after the examination starts.
3. この問題は全部で 10 ページ（表紙を除く）です。1～5 ページは日本語版、6～10 ページは英語版です。
This booklet consists of 10 pages, excluding the cover sheet. The Japanese version is shown on pages 1-5 and the English version on pages 6-10.
4. 解答用紙（罫線有り）を 2 枚、下書き用紙（白紙）を 1 枚配布します。
You are given two ruled answer sheets and one white draft sheet.
5. 問題は全部で 2 問あります。問題 I の解答を 1 枚目、問題 II の解答を 2 枚目の解答用紙に、必ず分けて記入すること。
There are two problems. Write the answers to Problem I on the first answer sheet and the answers to Problem II on the second answer sheet.
6. 解答用紙に解答を記述する際に、問題 I の解答か問題 II の解答かを必ず明記すること。
When writing the answers, clearly label the problem number on each answer sheet.

平成 2 7 年 8 月 2 0 日

問題 I の解答は 1 枚目の解答用紙に記入すること。問題 I の解答であることを明記すること。

問題 I C 言語で、整数のキューを表すデータ型 `queue` を操作する関数が以下のように与えられているとする。

```
queue newq(void);           空のキューを生成して返す。
unsigned sizeq(queue);      キューのサイズ (キュー内の要素数) を返す。
void enq(queue, int);       キューの末尾に整数を追加する。
int peekq(queue);           キューの先頭の整数を返す。キューは変化しない。
int deq(queue);             キューの先頭の整数を取り除いて返す。
ただし、空のキューに対する peekq, deq の値は未定義とする。
```

これらを用いて、マージソートによりキューに含まれる整数を昇順に並べ替える以下のプログラムを考える。

```
void split(queue q, queue subq[]) {
    int i = 0;
    subq[0] = newq();
    subq[1] = newq();
    while (sizeq(q) > 0) {
        enq(subq[i], deq(q));
        i = (A) ;
    }
}

queue merge(queue q1, queue q2) {
    queue q = newq();
    while (sizeq(q1) > 0 || sizeq(q2) > 0) {
        if (sizeq(q1) == 0) {
            enq(q, deq(q2));
        } else if (sizeq(q2) == 0) {
            enq(q, deq(q1));
        } else if (peekq(q1) < peekq(q2)) {
            enq(q, deq(q1));
        } else {
            enq(q, deq(q2));
        }
    }
    return q;
}
```

```

queue ms(queue q) {
    if (sizeq(q) <= 1) {
        return q;
    } else {
        queue subq[2];
        split(q, subq);
        return merge(ms(subq[0]), ms(subq[1]));
    }
}

queue sort(queue q) { return ms(q); }

```

- (1) 関数 `split` は、引数 `q` の要素を、先頭から順に 2 つのキュー `subq[0]`, `subq[1]` に交互に移動し、`q` を 2 つに分割する。変数 `i` の値が、`while` ループの繰り返しごとに交互に 0 と 1 になるように、(A) に入る適切な式を答えなさい。
- (2) 関数 `merge` は、昇順に並んだ整数が入っている 2 つのキュー `q1`, `q2` を受け取り、両方の要素を含んだ昇順のキューに統合して返す。この関数を、以下のように書き換えるとする。

```

static queue merge(queue q1, queue q2) {
    queue q = newq();
    while (sizeq(q1) > 0 || sizeq(q2) > 0) {
        if (sizeq(q1) == 0 || (B)) {
            enq(q, deq(q2));
        } else {
            enq(q, deq(q1));
        }
    }
    return q;
}

```

書き換える前と結果が同じになるように、(B) に入る適切な式を答えなさい。

- (3) サイズが 4 のキューを `sort` の引数として与えると、結果が返ってくるまでに、`newq` と `enq` は、それぞれ全部で何回呼び出されるか答えなさい。
- (4) n 個のランダムな整数が入ったキューを引数として `sort` を呼び出すとする。次の文について、文が正しければ T で答え、誤っていれば F で答えて誤っている理由を簡単に説明しなさい。
- (a) `sort` の実行が終了するまでに `enq` が呼び出される合計回数と `deq` が呼び出される合計回数は必ず等しい。
 - (b) `sort` の実行が終了するまでに `enq` が呼び出される合計回数の平均値の漸近的な上界は $O(n \log n)$ のオーダーである。
 - (c) `sort` の実行が終了するまでに `enq` が呼び出される合計回数の最悪値の漸近的な上界は $O(n \log n)$ のオーダーである。

- (d) n が正ならば、すべての `ms` の呼び出しについて、`ms` の引数のキューのサイズは正である。
- (e) `merge` の一回の呼び出しを考えた時、`merge` の `while` ループの本体が実行される回数は、`merge` の引数 `q1` と `q2` に入っている整数の値に依存する。
- (5) 配列を用いた循環バッファでキューを実装することを考える。以下で、`MAX` は十分に大きな定数で、`MAX` を超える個数の整数がキューに追加される場合は考えなくてよいとする。`malloc` によるメモリの割り付けは必ず成功するものとする。

```
typedef struct queue_head {
    unsigned count, head;
    int data[MAX];
} *queue;

queue newq(void) {
    queue q = malloc(sizeof(struct queue_head));
    q->count = q->head = 0;
    return q;
}

unsigned sizeq(queue q) { return q->count; }

void enq(queue q, int v) {
    q->data[ (C) ] = v;
    q->count++;
}

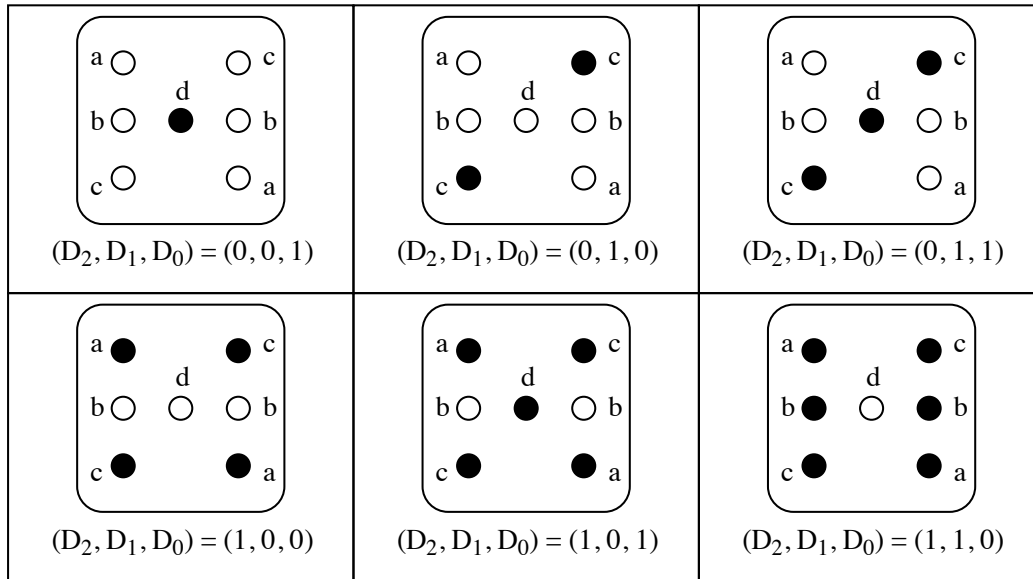
int peekq(queue q) { return q->data[q->head]; }

int deq(queue q) {
    int value = peekq(q);
    q->head = (q->head + 1) % MAX;
    (D) ;
    return value;
}
```

(C)、(D) に入る適切な式を答えなさい。

問題 II の解答は 2 枚目の解答用紙に記入すること。問題 II の解答であることを明記すること。

問題 II D_2, D_1, D_0 を入力として、 a, b, c, d を出力するデジタル回路を考える。以下の図のように、出力 a, b, c, d が LED に接続されており、サイコロの目を LED で表示する。●は点灯を示し、○は消灯を示す。LED は対応する出力が 1 のときに限り点灯し、0 のときに限り消灯するものとする。以下の設問に答えなさい。



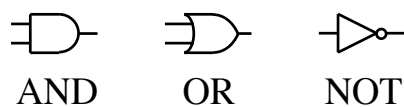
(1) D_2, D_1, D_0 を入力として、以下の真理値表を完成しなさい。x は don't care を意味する。

D_2	D_1	D_0	a	b	c	d
0	0	0	x	x	x	x
0	0	1	0	0	0	1
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1	x	x	x	x

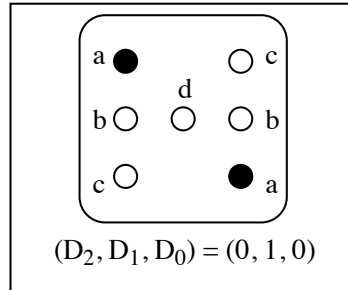
(2) 出力 a, b, c, d のそれぞれについてのカルノー図を示しなさい。

(3) (2) のカルノー図を使って、出力 a, b, c, d の最簡形の論理式を示しなさい。

(4) (3) の論理式を実現する回路図を示しなさい。ただし、以下の回路記号を必要に応じて用いなさい。
AND ゲートと OR ゲートは 2 入力とする。



- (5) $(D_2, D_1, D_0) = (0, 1, 0)$ のときだけ、サイコロの目を次のように変更する。そのための最簡形の論理式に基づいた回路図を示しなさい。



Write the answers to Problem I on the first answer sheet, and clearly label it at the top of the page as “Problem I.”

Problem I Suppose operations for a data type `queue`, which represents a queue of integers, are given as the following C functions.

<code>queue newq(void);</code>	Creates and returns an empty queue.
<code>unsigned sizeq(queue);</code>	Returns the size (i.e. number of elements) of this queue.
<code>void enq(queue, int);</code>	Appends an integer at the end of this queue.
<code>int peekq(queue);</code>	Returns the first integer of this queue without altering the queue itself.
<code>int deq(queue);</code>	Removes and returns the first integer of this queue.

The functions `peekq` and `deq` are not defined for an empty queue.

Let us consider the following program, which sorts integers of a queue in ascending order using merge sort.

```
void split(queue q, queue subq[]) {
    int i = 0;
    subq[0] = newq();
    subq[1] = newq();
    while (sizeq(q) > 0) {
        enq(subq[i], deq(q));
        i = (A) ;
    }
}

queue merge(queue q1, queue q2) {
    queue q = newq();
    while (sizeq(q1) > 0 || sizeq(q2) > 0) {
        if (sizeq(q1) == 0) {
            enq(q, deq(q2));
        } else if (sizeq(q2) == 0) {
            enq(q, deq(q1));
        } else if (peekq(q1) < peekq(q2)) {
            enq(q, deq(q1));
        } else {
            enq(q, deq(q2));
        }
    }
    return q;
}
```



```

queue ms(queue q) {
    if (sizeq(q) <= 1) {
        return q;
    } else {
        queue subq[2];
        split(q, subq);
        return merge(ms(subq[0]), ms(subq[1]));
    }
}

queue sort(queue q) { return ms(q); }

```

- (1) The function `split` divides elements of its argument `q` into two queues, moving the elements of `q` alternatively to `subq[0]` and `subq[1]`. The value of `i` should alternate between two values 0 and 1 for each iteration of `while`. Find an appropriate expression for (A) in the program.
- (2) The function `merge` receives two queues `q1` and `q2`, which both contain integers that have been already sorted in ascending order, and returns a queue which is also sorted in ascending order containing elements from both queues. Suppose we rewrite the function as follows:

```

static queue merge(queue q1, queue q2) {
    queue q = newq();
    while (sizeq(q1) > 0 || sizeq(q2) > 0) {
        if (sizeq(q1) == 0 || (B)) {
            enq(q, deq(q2));
        } else {
            enq(q, deq(q1));
        }
    }
    return q;
}

```

Find an appropriate expression for (B) so that it outputs the same result.

- (3) If we input a queue of size 4 as an argument to `sort`, how many times will `newq` and `enq` be called, respectively, before the result is returned?
- (4) Suppose we call `sort` with a queue containing n random integers. Answer T (true) for correct statement(s), and F (false) for incorrect statement(s). In addition, for each statement for which you answered F, briefly describe what is wrong with the statement.
 - (a) The total number of times `enq` is called before `sort` returns is always the same as the total number of times `deq` is called.
 - (b) The asymptotic upper bound of the total number of `enq` calls on average is on the order of $O(n \log n)$.

- (c) The asymptotic upper bound of the total number of `enq` calls in the worst case is on the order of $O(n \log n)$.
 - (d) If n is positive, then for all calls of `ms`, the size of the argument of `ms` is positive.
 - (e) For a call of `merge`, the number of times the body of the `while` loop is executed depends on the value of the integers in the arguments `q1` and `q2`.
- (5) Suppose we implement `queue` with a circular buffer using an array. In the following code, assume `MAX` to be a sufficiently large constant, and the number of integers in a queue to never exceed `MAX`. Also assume that the memory allocation with `malloc` always succeeds.

```
typedef struct queue_head {
    unsigned count, head;
    int data[MAX];
} *queue;

queue newq(void) {
    queue q = malloc(sizeof(struct queue_head));
    q->count = q->head = 0;
    return q;
}

unsigned sizeq(queue q) { return q->count; }

void enq(queue q, int v) {
    q->data[ (C) ] = v;
    q->count++;
}

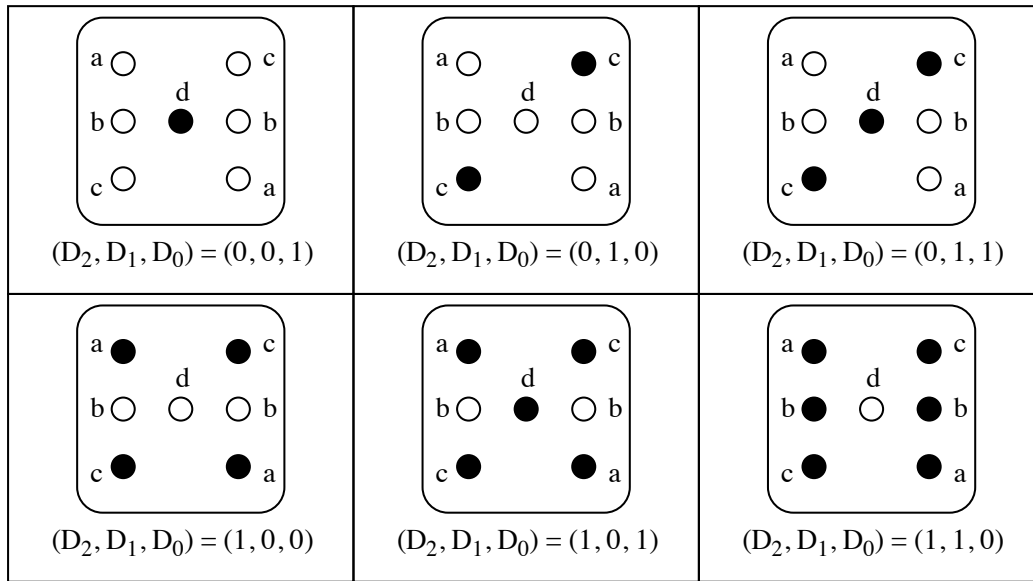
int peekq(queue q) { return q->data[q->head]; }

int deq(queue q) {
    int value = peekq(q);
    q->head = (q->head + 1) % MAX;
    (D) ;
    return value;
}
```

Find appropriate expressions for (C) and (D) in the program, respectively.

Write the answers to Problem II on the second answer sheet, and clearly label it at the top of the page as “Problem II.”

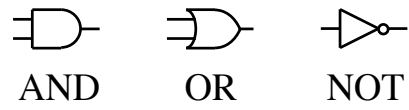
Problem II Let us consider a digital circuit with D_2, D_1 and D_0 as inputs and with a, b, c and d as outputs. As shown in the following figure, the outputs a, b, c and d are connected to LEDs and they indicate a dice spot. “●” means a LED is turned on, and “○” means a LED is turned off. LED lights up only when the corresponding output is “1” and lights off only for “0”. Answer the following questions.



- (1) Let D_2, D_1 and D_0 be input signals. Complete the following truth table. “x” means “don’t care.”

D_2	D_1	D_0	a	b	c	d
0	0	0	x	x	x	x
0	0	1	0	0	0	1
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1	x	x	x	x

- (2) Show the Karnaugh maps for each of the outputs a, b, c and d .
- (3) Describe the simplest logical expression for each of the outputs a, b, c and d from the Karnaugh maps in question (2).
- (4) Derive a schematic for realizing the logical expressions in question (3). Here, use the following symbols if necessary. AND and OR logic gates have two input terminals only.



- (5) Let us replace the dice spot only for $(D_2, D_1, D_0) = (0, 1, 0)$ with the following figure. Derive a schematic based on the simplest logical expression.

